

Απαλλακτική Εργασία — Αναφορά

Νεφοϋπολογιστική και Προηγμένες Αρχιτεκτονικές Εφαρμογών

Θέμα: Word Game — Microservices με Docker Compose και Low-Code UI (Appsmith)

Σπουδαστής/ές: _____

Ημερομηνία: 2025-12-27

Repository: word-game-complete

Σύνοψη

- Η εφαρμογή είναι ένα playable word-guessing game με REST backend (FastAPI) και UI σε Appsmith.
- Το σύστημα εκκινεί ταυτόχρονα μέσω Docker Compose και περιλαμβάνει τα 5 υποχρεωτικά containers του brief, καθώς και ένα επιπλέον container (backend) για την υλοποίηση της λογικής του παιχνιδιού.
- Χρησιμοποιείται PostgreSQL για persistence (≥ 2 tables) και Adminer για διαχείριση/επαλήθευση.
- Υλοποιούνται 2 mock REST APIs: (1) `svenwall/jsonplaceholder` (volume-seeded), (2) custom Dockerfile (`json-server`).
- Υπάρχει εξωτερική (serverless) διασύνδεση με δημόσιο API (`random-word-api.herokuapp.com`) και καταγραφή των κλήσεων στη βάση.

Κεφάλαιο 1 — Εισαγωγή και περιγραφή σεναρίου

Η θεματολογία που επιλέχθηκε είναι ένα “Word Game” όπου ο χρήστης ξεκινά ένα νέο παιχνίδι, προσπαθεί να μαντέψει μια “target” λέξη, βλέπει semantic similarity scores για τις προσπάθειες, και μπορεί να ζητήσει hints. Η επιλογή αυτής της θεματολογίας επιτρέπει να υλοποιηθούν καθαρά UI flows (λήψη δεδομένων από APIs, επιλογή από χρήστη, αποθήκευση στη DB) και παράλληλα να είναι ένα πλήρως λειτουργικό deliverable.

Βασικές ροές χρήστη (από το UI)

- Start new game (με player name + προαιρετικό word length).
- Submit guess και αποθήκευση κάθε guess στη βάση.
- Get hint (με προαιρετική επιλογή hint type από Mock API).
- Reveal word (τερματισμός session).
- Προβολή leaderboard και logs (sessions, guesses, API logs).

Κεφάλαιο 2 — Απαιτήσεις άσκησης και αντιστοίχιση υλοποίησης

Οι υποχρεωτικές απαιτήσεις της άσκησης περιγράφονται στο `apalaktiki\_ntinos.pdf`. Ο παρακάτω πίνακας δείχνει πώς καλύπτονται από τα services του `compose.yaml`.

Απαίτηση (brief)	Υλοποίηση σε αυτό το repo
Low <code>code</code> UI	Appsmith container (`appsmith/appsmith-ce`) + export `appsmith/word-game-app`
Local DB (>=2 tables)	Postgres container + schema `sql/init.sql` + sample rows `sql/export.sql`
Local DB admin	Adminer container (web UI στο `http://localhost:8080`)
Mock REST API #1 (svenwal/jsonplaceholder)	`mock-api-1` seeded απ <code>rest-apis/mock-api-1/db.json` (volume)</code>
Mock REST API #2 (custom Dockerfile)	`mock-api-2` built απ <code>rest-apis/mock-api-2/Dockerfile` + `db.json`</code>
Serverless DB/API integration	External API `random-word-api.herokuapp.com` + logging στο `api_logs`

Σημείωση: Το brief μιλά για “5 containers” ως ελάχιστο υποχρεωτικό σύνολο. Το συγκεκριμένο repo χρησιμοποιεί 6 containers (τα 5 υποχρεωτικά + ένα backend) ώστε η εφαρμογή να έχει πραγματική λογική/κανόνες και να μην είναι απλά demo APIs.

Κεφάλαιο 3 — Αρχιτεκτονική συστήματος

Το σύστημα υλοποιείται ως μικρή microservices αρχιτεκτονική σε Docker Compose. Η επικοινωνία μεταξύ containers γίνεται μέσω του private network που δημιουργεί το Compose (Docker DNS service names).

Services (όπως ορίζονται στο compose.yaml)

Service	Container name	Image / Build	Ports (host:container)
appsmith	word-game-ui	appsmith/appsmith-ce:latest	80:80
postgres	word-game-db	postgres:15-alpine	5432:5432
adminer	word-game-admin	adminer:latest	8080:8080
backend	word-game-backend	build: ./backend (Dockerfile)	(not published)
mock-api-2	mock-hints	build: ./rest-apis/mock-api-2 (Dockerfile)	3001:3001
mock-api-1	mock-wordpacks	svenwal/jsonplaceholder:latest	3000:3000

Host vs Docker network (σημαντικό για Appsmith datasources)

- Από τον browser του host χρησιμοποιούμε `http://localhost:PORT` (π.χ. `http://localhost:3000/wordpacks`).
- Μέσα από Appsmith (container) το `localhost` σημαίνει “ο ίδιος ο container”, άρα αποτυγχάνει.
- Στο Appsmith πρέπει να δηλώνονται service names: `http://mock-api-1:3000`, `postgres:5432`, `http://backend:8000`.

Κεφάλαιο 4 — Υλοποίηση (UI, Backend, DB, APIs)

4.1 Appsmith (Low-code UI)

Το UI υλοποιείται στο Appsmith και παραδίδεται ως export αρχείο: ``appsmith/word-game-app.json``. Μετά την εισαγωγή, ο χρήστης πρέπει να “Reconnect” τις datasources (credentials/URLs) γιατί το Appsmith δεν μεταφέρει μυστικά (passwords) μέσα στο export.

Datasources που απαιτούνται

Datasource	Type (Appsmith)	Σκοπός
------------	-----------------	--------

4.2 FastAPI backend

Το backend τρέχει ως container (``backend``) και εκθέτει REST endpoints που καλεί το Appsmith. Η ενεργή κατάσταση παιχνιδιού κρατιέται in-memory ανά ``player_name`` (active target word + guesses), ενώ τα ιστορικά δεδομένα γράφονται στη βάση (Postgres).

Ενδεικτικά endpoints

- POST ``/start`` — δημιουργεί νέο game session και επιλέγει target word (wordpack ή external ή local fallback).
- POST ``/guess`` — καταγράφει guess και επιστρέφει similarity score + win state.
- POST ``/hint`` — επιστρέφει hint (επηρεάζεται από επιλεγμένο hint type στο UI).
- POST ``/reveal`` — αποκαλύπτει τη λέξη και κλείνει το session.
- GET ``/leaderboard`` — υπολογίζει/επιστρέφει leaderboard με βάση finished sessions.

4.3 PostgreSQL schema

Το schema δημιουργείται από ``sql/init.sql`` και φορτώνονται αρχικά sample rows από ``sql/export.sql`` μόνο στο πρώτο boot (όταν ο docker volume είναι άδειος).

Πίνακες

- ``game_sessions``
- ``guesses``
- ``leaderboard``
- ``player_stats``
- ``favorites``
- ``api_logs``

4.4 Mock APIs + External API

- Mock API 1 (``mock-api-1``): wordpacks dataset (``/wordpacks``), βασισμένο στο ``svenwal/jsonplaceholder``.
- Mock API 2 (``mock-api-2``): hint types dataset (``/hints``), χτισμένο από custom Dockerfile με ``json-server``.
- External API: ``random-word-api.herokuapp.com`` (serverless). Χρησιμοποιείται ως default source για target word και οι κλήσεις καταγράφονται στον πίνακα ``api_logs``.

Κεφάλαιο 5 — Εγκατάσταση και εκτέλεση

Βήματα εκκίνησης (Docker Compose)

```
docker compose -f compose.yaml up -d --build
docker compose -f compose.yaml ps
```

URLs (από τον browser του host)

- Appsmith: `http://localhost`
- Adminer: `http://localhost:8080`
- Mock API 1: `http://localhost:3000/wordpacks`
- Mock API 2: `http://localhost:3001/hints`

Appsmith import + datasource reconnect

- Open Appsmith στο `http://localhost` και δημιουργήστε local admin user (δεν απαιτείται cloud account).
- Import το `appsmith/word-game-app.json`.
- Reconnect datasources (σημαντικό: μέσα στο Appsmith χρησιμοποιούμε service names, όχι `localhost`).
- Postgres datasource: host `postgres`, port `5432`, db `wordgame`, user `gameuser`, password `gamepass123`, SSL disabled.
- REST datasources: `http://mock-api-1:3000`, `http://mock-api-2:3001`, `http://backend:8000`, `https://random-word-api.herokuapp.com`.

Clean reset (όταν θέλετε “καθαρό” Appsmith + DB)

```
docker compose -f compose.yaml down -v
docker compose -f compose.yaml up -d --build
```

Κεφάλαιο 6 — Evidence / Screenshots για αξιολόγηση

Το brief ζητά παρουσίαση (ppt) και αναφορά (pdf) με οδηγίες εγκατάστασης/λειτουργίας και screenshots. Παρακάτω είναι τα προτεινόμενα screenshots που αποδεικνύουν ότι καλύπτονται όλες οι απαιτήσεις.

6.1 Docker Compose (όλα τα containers Up)

INSERT SCREENSHOT

Command: `docker compose -f compose.yaml ps`

Expected: appsmith, postgres, adminer, mock-api-1, mock-api-2, backend are Up

6.2 Appsmith UI (Gameplay + Integrations & Logs)

INSERT SCREENSHOT

Gameplay tab: start new game + submit guesses + leaderboard

Integrations & Logs tab: wordpacks + hint types + logs tables

6.3 Adminer (DB persistence)

INSERT SCREENSHOT

Adminer: show game_sessions + guesses populated after a game
Also show api_logs populated after a Start Game using external API

6.4 Mock APIs (browser JSON)

INSERT SCREENSHOT

<http://localhost:3000/wordpacks> and <http://localhost:3001/hints> in browser

Κεφάλαιο 7 — Συμπεράσματα

Η υλοποίηση καλύπτει τις απαιτήσεις του brief: Compose orchestration, Low-code UI, local DB + admin UI, δύο mock REST APIs με τους απαιτούμενους τρόπους δημιουργίας, και external serverless API integration. Η προσθήκη του FastAPI backend επιτρέπει το σύστημα να είναι ένα πλήρως λειτουργικό παιχνίδι, ώστε οι ροές στο UI να είναι meaningful και να παράγουν πραγματικά δεδομένα στη βάση.